TASK 1: EXECUTE THE SCRIPTS SHOWN IN THE VIDEO

Task: Create a Private Amazon S3 Bucket and DynamoDB Table Using Terraform

1. Objective

To create a private Amazon S3 bucket and an Amazon DynamoDB table using Terraform. The S3 bucket is configured to block public access, and the DynamoDB table is configured with a partition key named LockID for learning Terraform state-locking concepts.

2. Purpose

Infrastructure as Code (IaC) allows AWS resources to be created, configured, and managed using configuration files instead of creating each resource manually through the AWS Management Console.

In this task:

- **Amazon S3:** Stores objects and can be used to store Terraform remote state.
- **S3 Public Access Block:** Prevents public access through bucket ACLs and policies.
- **Amazon DynamoDB:** Provides a table structure suitable for Terraform state-locking exercises.
- **Terraform:** Defines and provisions the AWS resources through HCL configuration.

Important: This configuration creates the S3 bucket and DynamoDB table, but it does not yet configure the S3 bucket as a Terraform backend or connect the DynamoDB table to a backend.

3. Terraform Configuration

Create a file named **main.tf** and add the following code.

```
resource "aws_s3_bucket" "s3bucket" {
  bucket = "s3-backend-terraform-567487919764"

  tags = {
    Name = "Private-S3-Backend"
  }
}

resource "aws_s3_bucket_public_access_block" "s3bucket" {
  bucket = aws_s3_bucket.s3bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}

resource "aws_dynamodb_table" "dynamodb_terraform_state_lock" {
  name         = "terraform-state-lock-dynamodb"
  hash_key     = "LockID"
  read_capacity  = 20
```

```
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

5. Explanation of the Code

5.1 Create the S3 Bucket

```
resource "aws_s3_bucket" "s3bucket" {
  bucket = "s3-backend-terraform-567487919764"

  tags = {
    Name = "Private-S3-Backend"
  }
}
```

- resource: Declares an AWS resource managed by Terraform.
- aws_s3_bucket: Identifies the AWS provider resource type for an S3 bucket.
- s3bucket: The Terraform-local name for this resource.
- bucket: Specifies the globally unique S3 bucket name.
- tags: Adds a descriptive name to the bucket.

S3 bucket names must follow AWS naming rules and be globally unique.

5.2 Block Public Access

```
resource "aws_s3_bucket_public_access_block" "s3bucket" {
  bucket = aws_s3_bucket.s3bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

This resource configures all four S3 Block Public Access settings.

- block_public_acls: Blocks new public ACLs.
- ignore_public_acls: Ignores public ACL permissions.
- block_public_policy: Blocks public bucket policies.
- restrict_public_buckets: Restricts access granted by public bucket policies.

The expression aws_s3_bucket.s3bucket.id references the bucket created by the first resource. Terraform automatically determines that the bucket must exist before it can configure its public-access settings.

These settings help keep the bucket private, assuming no other access mechanism grants unintended access.

5.3 Create the DynamoDB Table

```
resource "aws_dynamodb_table" "dynamodb_terraform_state_lock" {
  name         = "terraform-state-lock-dynamodb"
  hash_key     = "LockID"
  read_capacity  = 20
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

- aws_dynamodb_table: Identifies the DynamoDB table resource.
- name: Sets the table name to terraform-state-lock-dynamodb.
- hash_key: Defines LockID as the partition key.
- read_capacity: Configures 20 read capacity units per second.
- write_capacity: Configures 20 write capacity units per second.
- attribute: Declares the partition-key attribute.
- type = "S": Specifies that LockID is a String.

Because this configuration uses provisioned capacity, AWS charges are based on the configured capacity and applicable pricing. For a small practice lab, on-demand billing may be more convenient.

6. Implementation Steps

Step 1: Open the project directory

Open Git Bash and navigate to your Terraform project directory.
Make sure that main.tf is saved in this directory.

Step 2: Format the configuration terraform fmt

Step 3: Initialize Terraform terraform init

Step 4: Validate the configuration terraform validate

Expected result: Success! The configuration is valid.

Step 5: Review the execution plan terraform plan

Review the planned resources before applying the configuration. Terraform should plan to create the S3 bucket, its public-access-block configuration, and the DynamoDB table if they do not already exist.

Step 6: Create the resources → terraform apply
Review the plan and type: yes

8. Implementation

main.tf

```
resource "aws_s3_bucket" "s3bucket" {
  bucket = "s3-backend-terraform-567487919764"

  tags = {
    Name = "Private-S3-Backend"
  }
}

resource "aws_s3_bucket_public_access_block" "s3bucket" {
  bucket = aws_s3_bucket.s3bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}

resource "aws_dynamodb_table" "dynamodb_terraform_state_lock" {
  name           = "terraform-state-lock-dynamodb"
  hash_key       = "LockID"
  read_capacity  = 20
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

Problems Output Debug Console Terminal

```
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
      + restrict_public_buckets = true
    }

Plan: 2 to add, 0 to change, 0 to destroy.

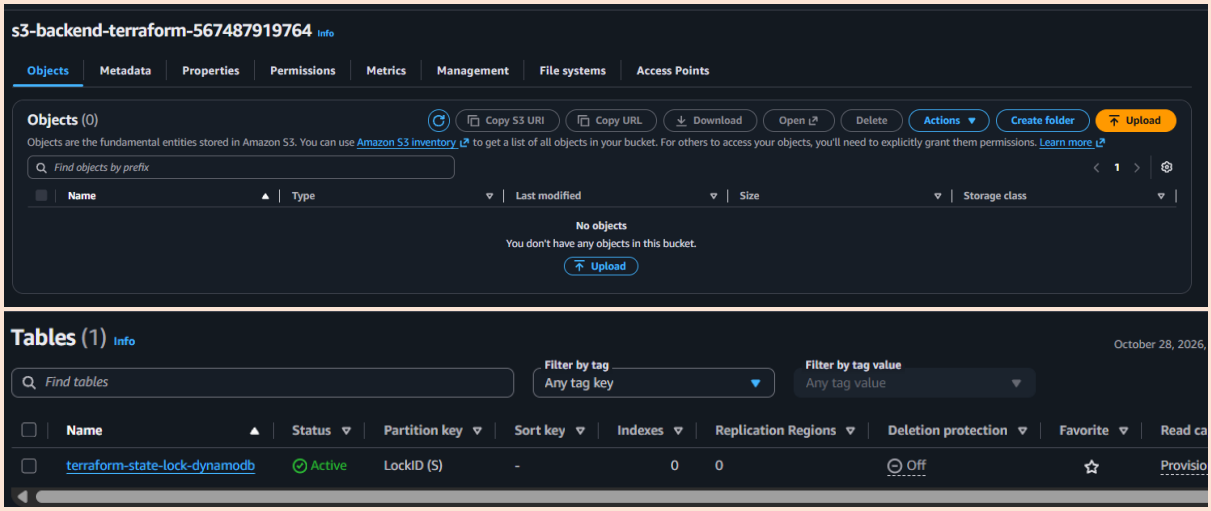
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_bucket.s3bucket: Creating...
aws_s3_bucket.s3bucket: Creation complete after 6s [id=s3-backend-terraform-567487919764]
aws_s3_bucket_public_access_block.s3bucket: Creating...
aws_s3_bucket_public_access_block.s3bucket: Creation complete after 1s [id=s3-backend-terraform-567487919764]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$
```



9. Conclusion

Successfully defined an Amazon S3 bucket, configured S3 Block Public Access, and defined a DynamoDB table using Terraform.

This task demonstrates how Terraform can provision AWS storage and database resources consistently through Infrastructure as Code. The S3 bucket is configured to block public access, while the DynamoDB table provides the key structure needed for further state-locking exercises.

TASK: CONFIGURE AMAZON S3 AS A TERRAFORM REMOTE BACKEND

1. Objective

To configure Amazon S3 as a remote backend for Terraform and store the Terraform state file (**terraform.tfstate**) in an S3 bucket instead of relying only on a local state file.

2. Purpose

Terraform uses a state file to track the AWS infrastructure it manages. By default, Terraform stores the state locally in the project directory.

Using Amazon S3 as a remote backend provides centralized state storage and supports collaboration between users working with the same infrastructure.In this task:

- Amazon S3 stores the Terraform state file.
- The key parameter specifies the state file's location inside the bucket.
- The region parameter specifies the AWS region.
- The encrypt parameter enables encryption for the remote state object.
- The dynamodb_table parameter configures the legacy DynamoDB-based locking mechanism.

3. Prerequisites

1. An AWS account.
2. Terraform installed on the local machine.
3. AWS CLI configured with suitable permissions.
4. An existing S3 bucket.
5. An existing DynamoDB table named terraform-state-lock-dynamodb.
6. The DynamoDB table must have a partition key named LockID of type String.
7. The S3 bucket and DynamoDB table must be in the us-east-1 region for this configuration.

4. Resources Used

Resource	Name
S3 bucket	s3-backend-terraform-567487919764
State file key	terraform.tfstate
AWS region	us-east-1
DynamoDB table	terraform-state-lock-dynamodb

5. Implementation Steps

Step 1: Create the Terraform configuration

Open the project directory in VS Code and add the following backend configuration to a Terraform file, such as **main.tf**.

```
terraform {
  backend "s3" {
    bucket       = "s3-backend-terraform-567487919764"
    key          = "terraform.tfstate"
    region       = "us-east-1"
    dynamodb_table = "terraform-state-lock-dynamodb"
    encrypt      = true
  }
}
```

Step 2: Understand the configuration

- backend "s3": Configures Amazon S3 as the remote state backend.
- bucket: Specifies the existing S3 bucket.
- key: Defines the location of the state object inside the bucket.
- region: Specifies the AWS region.
- dynamodb_table: Specifies the DynamoDB table for legacy Terraform state locking.
- encrypt = true: Enables encryption for the remote state object.

Because the key is terraform.tfstate, the state object is stored at the root of the specified S3 bucket for the default workspace.

Step 3: Initialize the backend

Open Git Bash in the project directory and run: terraform init

Terraform initializes the S3 backend.

If you are migrating an existing local state file to S3, Terraform may ask whether to copy the existing state to the new backend. Review the prompt and confirm the migration if that is your intention.

If the backend configuration has already been initialized and you intentionally changed its settings, follow Terraform's backend reconfiguration or migration instructions as appropriate.

Step 4: Verify the remote state file

Open the AWS Management Console.

1. Navigate to Amazon S3.
2. Open the bucket s3-backend-terraform-567487919764.
3. Select the **Objects** tab.
4. Look for terraform.tfstate.

Step 5: Verify Terraform state access: terraform state list

If your configuration manages resources and the state is accessible, Terraform displays the resource addresses tracked in the state.

Next, run: terraform plan

Terraform should successfully access the remote state and compare the configuration with the actual infrastructure, assuming the configuration and AWS permissions are valid.

7. Implementation

```
terraform {
  backend "s3" {
    bucket         = "s3-backend-terraform-567487919764"
    key            = "terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-state-lock-dynamodb"
    encrypt        = true
  }
}
```

```
Problems  Output  Debug Console  Terminal

Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform validate
Success! The configuration is valid.


Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply

  Warning: Deprecated Parameter

  The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

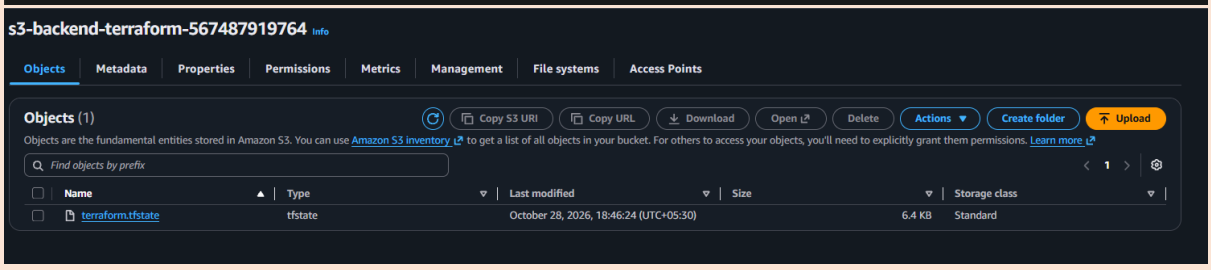
Acquiring state lock. This may take a few moments...
aws_dynamodb_table.dynamodb_terraform_state_lock: Refreshing state... [id=terraform-state-lock-dynamodb]
aws_s3_bucket.s3bucket: Refreshing state... [id=s3-backend-terraform-567487919764]
aws_s3_bucket_public_access_block.s3bucket: Refreshing state... [id=s3-backend-terraform-567487919764]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Releasing state lock. This may take a few moments...

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$
```



8. Conclusion

Successfully configured Amazon S3 as the Terraform remote backend. The terraform.tfstate object was visible in the specified S3 bucket, providing evidence that the state file was stored remotely.

Remote state storage helps centralize Terraform state and supports collaboration. The backend can also provide state locking when configured with a supported locking mechanism.

Terraform Provisioners — Local, Remote and Provisioner Behaviors

local-exec — runs a command on the machine where Terraform is executed, such as your Windows PC running VS Code and Git Bash.

remote-exec — connects to a remote machine, usually an EC2 instance, and runs commands on that machine over SSH or WinRM.

1. Objective

To understand and implement Terraform provisioners using practical examples, including:

1. local-exec — execute commands on the local machine.
2. remote-exec — execute commands on a remote EC2 instance.
3. Default creation-time provisioner behavior.
4. Destroy-time provisioner behavior using when = destroy.
5. Provisioner failure handling using on_failure = fail.
6. Provisioner failure handling using on_failure = continue.

2. Purpose

Terraform is an Infrastructure as Code (IaC) tool used to create, modify and manage infrastructure.

Provisioners allow Terraform to run commands or scripts on the local machine or a remote resource during its lifecycle.

For example:

- Create a local log file after Terraform creates a resource.
- Run a cleanup command before Terraform destroys a resource.
- Install Apache on an EC2 instance using SSH.
- Decide whether Terraform should stop or continue when a provisioner command fails.

HashiCorp recommends using provisioners only when a suitable alternative is unavailable, because Terraform cannot fully model the effects of arbitrary scripts. Official documentation.

For the local-exec lab

No AWS resources are required. This lab uses Terraform's local provider to create a local file.

For the remote-exec lab

- An AWS account.
- AWS CLI credentials configured.
- AWS region us-east-1.
- An existing EC2 key pair.
- The corresponding private key file.
- A default VPC and subnet with internet connectivity.
- A security group allowing SSH from your public IP and HTTP traffic on port 80.

Security: Never upload your .pem private key to GitHub or share it publicly.

---

3. Task 1 — Understand local-exec

Objective

Execute a command on the local computer where Terraform runs.

Purpose

The local-exec provisioner is useful for local logging, running scripts, or invoking local commands after a resource is created.

Step 1: Create the project folder

Create the following folder and open it in VS Code: terraform-local-exec/

Step 2: Create main.tf

Copy the following configuration:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform local-exec demo"

  provisioner "local-exec" {
    command = "echo Local provisioner executed >> provisioner-demo.txt"
  }
}
```

Step 3: Initialize Terraform

Open the VS Code terminal, select Git Bash, and execute the commands from inside the project folder:

→ terraform init
→ terraform validate
→ terraform plan
→ terraform apply
Type yes when Terraform asks for confirmation.

Step 4: Verify the output

cat demo.txt cat provisioner-demo.txt

Expected output from demo.txt: Terraform local-exec demo

Expected output from provisioner-demo.txt: Local provisioner executed

Explanation

- resource "local_file" "demo" creates a local file.
- filename specifies where Terraform creates the file.
- content defines the file contents.
- provisioner "local-exec" runs a command on the local computer.
- >> appends the command output to a file.

Result: The local file resource is created, and the local provisioner executes its command.

---

4. Task 2 — Understand remote-exec

Objective

Execute commands on a remote Amazon EC2 instance using SSH.

Purpose

The remote-exec provisioner is useful for running commands on a remote server, such as installing packages or configuring a service.

Step 1: Create the project folder

Create and open: terraform-remote-exec/

Step 2: Create main.tf

Replace the placeholder values before running this configuration.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

variable "key_name" {
  default = "YOUR_KEY_PAIR_NAME"
}

variable "private_key_path" {
  default = "C:/Users/YOUR_USER/Downloads/YOUR_KEY.pem"
}

variable "my_ip" {
```

```
  description = "Your public IPv4 address with /32"
  type        = string
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
}

data "aws_vpc" "default" {
  default = true
}

resource "aws_security_group" "web_sg" {
  name   = "terraform-remote-exec-sg"
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
resource "aws_instance" "web" {
  ami                    = data.aws_ami.amazon_linux.id
  instance_type          = "t3.micro"
  key_name               = var.key_name
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  associate_public_ip_address = true

  provisioner "remote-exec" {
    inline = [
      "sudo dnf install -y httpd",
      "sudo systemctl enable --now httpd",
      "echo 'Hello from Terraform remote-exec' | sudo tee /var/www/html/index.html"
    ]

    connection {
      type        = "ssh"
      user        = "ec2-user"
      private_key = file(var.private_key_path)
      host        = self.public_ip
    }
  }

  tags = {
    Name = "Terraform-Remote-Exec"
  }
}

output "public_ip" {
  value = aws_instance.web.public_ip
}

output "website_url" {
  value = "http://${aws_instance.web.public_ip}"
}
```

Step 3: Supply your actual values

Update:

- YOUR_KEY_PAIR_NAME with your existing EC2 key pair name.
- YOUR_USER with your Windows user-folder name.
- YOUR_KEY.pem with your actual private-key filename.

The my_ip variable must contain your public IPv4 address followed by /32, for example 203.0.113.10/32 (replace this example with your real IP).

You can create a terraform.tfvars file in the same folder:

```
key_name        = "your-actual-key-pair"
private_key_path = "C:/Users/YourUser/Downloads/your-key.pem"
my_ip           = "YOUR_ACTUAL_PUBLIC_IP/32"
```

Do not commit the .tfvars file if it contains sensitive information.

Step 4: Initialize and apply

Run from the terraform-remote-exec folder:

→ terraform init
→ terraform validate
→ terraform plan
→ terraform apply
Type yes when prompted.

Terraform creates the EC2 instance, waits for SSH connectivity, and runs the commands through remote-exec.

5. Task 3 — Default provisioner behavior

Objective

Understand when Terraform executes a provisioner if when is not specified.

Example

```
resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform provisioner demo"

  provisioner "local-exec" {
    command = "echo Resource created >> provisioner.log"
  }
}
```

Behavior

By default, Terraform executes the provisioner during resource creation, after the resource has been created.

The when argument is omitted because creation-time execution is the default.

Execute

→ terraform init
→ terraform apply
Verify

cat provisioner.log

Expected output: Resource created

Important: Terraform does not rerun a creation-time provisioner on every terraform apply. If the resource has no changes requiring creation, the provisioner normally does not run again.

---

6. Task 4 — Destroy-time provisioner

Objective

Execute a command when Terraform destroys a resource.

Example

```
resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform local-exec demo"

  provisioner "local-exec" {
    when    = destroy
    command = "echo Resource destroyed >> provisioner-deleted.txt"
  }
}
```

Behavior

The provisioner runs before Terraform removes the resource.

Execute

For a fresh lab, run:

→ terraform init
→ terraform apply
Then destroy the resource: terraform destroy

Type yes when prompted.

Verify

cat provisioner-deleted.txt

Expected output: Resource destroyed

The demo.txt file should be removed by Terraform, while provisioner-deleted.txt remains because it was created by the shell command and is not managed by this Terraform resource.

Important observations

- Keep the resource block and destroy provisioner in the configuration when running terraform destroy.
- Destroy-time provisioners do not run for tainted resources.
- The create_before_destroy lifecycle setting prevents destroy-time provisioners from running.
- Avoid relying on destroy provisioners for essential cleanup because their execution has lifecycle limitations.

---

7. Task 5 — on_failure = fail

Objective

Stop the Terraform operation when a provisioner command fails.

Example

```
resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform failure test"

  provisioner "local-exec" {
    command    = "exit 1"
    on_failure = fail
  }
}
```

Behavior

The command exit 1 deliberately returns a failure status.
With on_failure = fail:

1. The provisioner reports an error.
2. The Terraform operation fails.
3. A creation-time provisioner failure generally taints the resource so Terraform can replace it during a subsequent apply.

The fail setting is the default, so specifying it explicitly is optional.

Execute terraform apply
Observe the provisioner error.

Important: The provider may already have created the resource before the provisioner fails. A failed apply does not guarantee that no resource exists.

---

8. Task 6 — on_failure = continue

Objective
Allow Terraform to continue when a provisioner command fails.

Example

```
resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform continue test"

  provisioner "local-exec" {
    command    = "exit 1"
    on_failure = continue
  }
}
```

Behavior

The command fails, but Terraform ignores that provisioner error and continues with the remaining operations.

Execute
→ terraform init
→ terraform apply

Observe the output. Terraform should not fail the overall apply solely because this provisioner command failed.

Important distinction

on_failure = continue does not reverse resource changes or guarantee that the script's intended task was completed. It only changes Terraform's response to the provisioner failure.

---

9. Comparison of provisioner behaviors

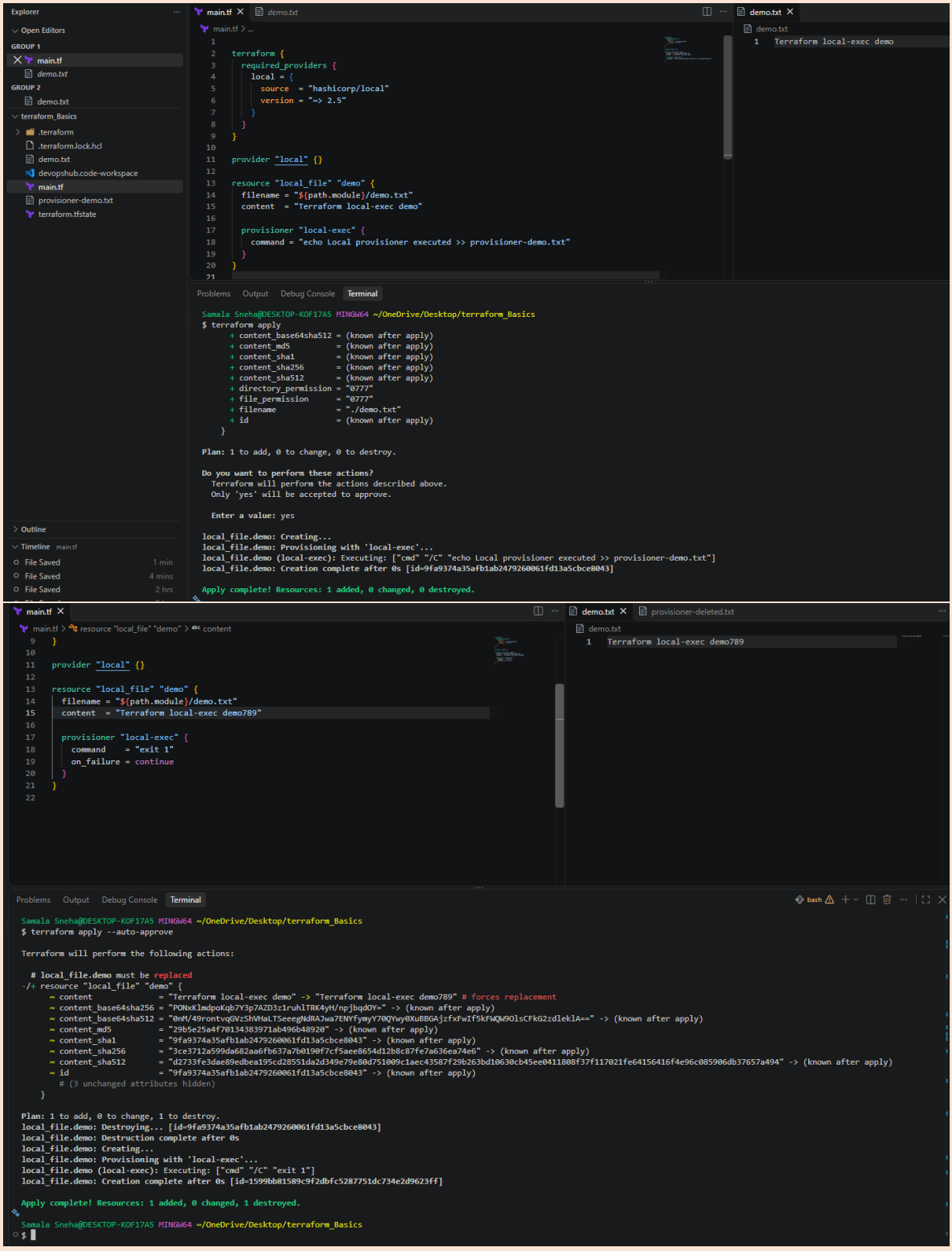
Behavior	Configuration	Meaning
Default creation-time	No when argument	Run after resource creation
Destroy-time	when = destroy	Run before resource destruction
Stop on failure	on_failure = fail	Fail the operation if the provisioner fails
Continue on failure	on_failure = continue	Ignore the provisioner error and continue

Local versus remote execution

Feature	local-exec	remote-exec
Execution location	Local machine	Remote machine
SSH required	No	Usually yes for Linux EC2
Main argument	command	inline, script or scripts
Typical use	Logging, local scripts	Server configuration
AWS resources required	Not for the local-file example	Yes, for the EC2 example

---

10.Implementation



```
Explorer
Open Editors
GROUP 1
main.tf
demo.txt
GROUP 2
demo.txt
terraform_Basics
.terraform
.terraform.lock.hcl
demo.txt
devopshub.code-workspace
main.tf
provisioner-demo.txt
terraform.tfstate
Outline
Timeline main.tf
File Saved 1 min
File Saved 4 mins
File Saved 2 hrs
```

main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform local-exec demo"

  provisioner "local-exec" {
    command = "echo Local provisioner executed >> provisioner-demo.txt"
  }
}
```

demo.txt

```
Terraform local-exec demo
```

Problems Output Debug Console Terminal

```
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./demo.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.demo: Creating...
local_file.demo: Provisioning with 'local-exec'...
local_file.demo (local-exec): Executing: ["cmd" "/C" "echo Local provisioner executed >> provisioner-demo.txt"]
local_file.demo: Creation complete after 0s [id=9fa9374a35afb1ab2479260061fd13a5cbce8043]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

main.tf

```
}

provider "local" {}

resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform local-exec demo789"

  provisioner "local-exec" {
    command    = "exit 1"
    on_failure = continue
  }
}
```

demo.txt

```
Terraform local-exec demo789
```

provisioner-deleted.txt

Problems Output Debug Console Terminal

```
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply --auto-approve

Terraform will perform the following actions:

  # local_file.demo must be replaced
-/+ resource "local_file" "demo" {
      ~ content              = "Terraform local-exec demo" -> "Terraform local-exec demo789" # forces replacement
      ~ content_base64sha256 = "PONxKlmdpoKqb7Y3p7AZD3z1ruhlTRK4yH/npjbqdOY=" -> (known after apply)
      ~ content_base64sha512 = "0nM/49rontvqGVzShVHaLTSeeegNdRAJwa7ENYfymyY70QYwy0XuBBGAjzfxFwIf5kFWQW9OlsCFkG2zdleklA==" -> (known after apply)
      ~ content_md5          = "29b5e25a4f70134383971ab496b48920" -> (known after apply)
      ~ content_sha1         = "9fa9374a35afb1ab2479260061fd13a5cbce8043" -> (known after apply)
      ~ content_sha256       = "3ce3712a599da682aa6fb637a7b0190f7cf5aee8654d12b8c87fe7a636ea74e6" -> (known after apply)
      ~ content_sha512       = "d2733fe3dae89edbea195cd28551da2d349e79e80d751009c1aec43587f29b263bd10630cb45ee0411808f37f117021fe64156416f4e96c085906db37657a494" -> (known after apply)
      ~ id                   = "9fa9374a35afb1ab2479260061fd13a5cbce8043" -> (known after apply)
        # (3 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.
local_file.demo: Destroying... [id=9fa9374a35afb1ab2479260061fd13a5cbce8043]
local_file.demo: Destruction complete after 0s
local_file.demo: Creating...
local_file.demo: Provisioning with 'local-exec'...
local_file.demo (local-exec): Executing: ["cmd" "/C" "exit 1"]
local_file.demo: Creation complete after 0s [id=1599bb81589c9f2dbfc5287751dc734e2d9623ff]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$
```

main.tf

```
}

provider "local" {}

resource "local_file" "demo" {
  filename = "${path.module}/demo.txt"
  content  = "Terraform local-exec demo"

  provisioner "local-exec" {
    command    = "exit 1"
    on_failure = fail
  }
}
```

Problems Output Debug Console Terminal

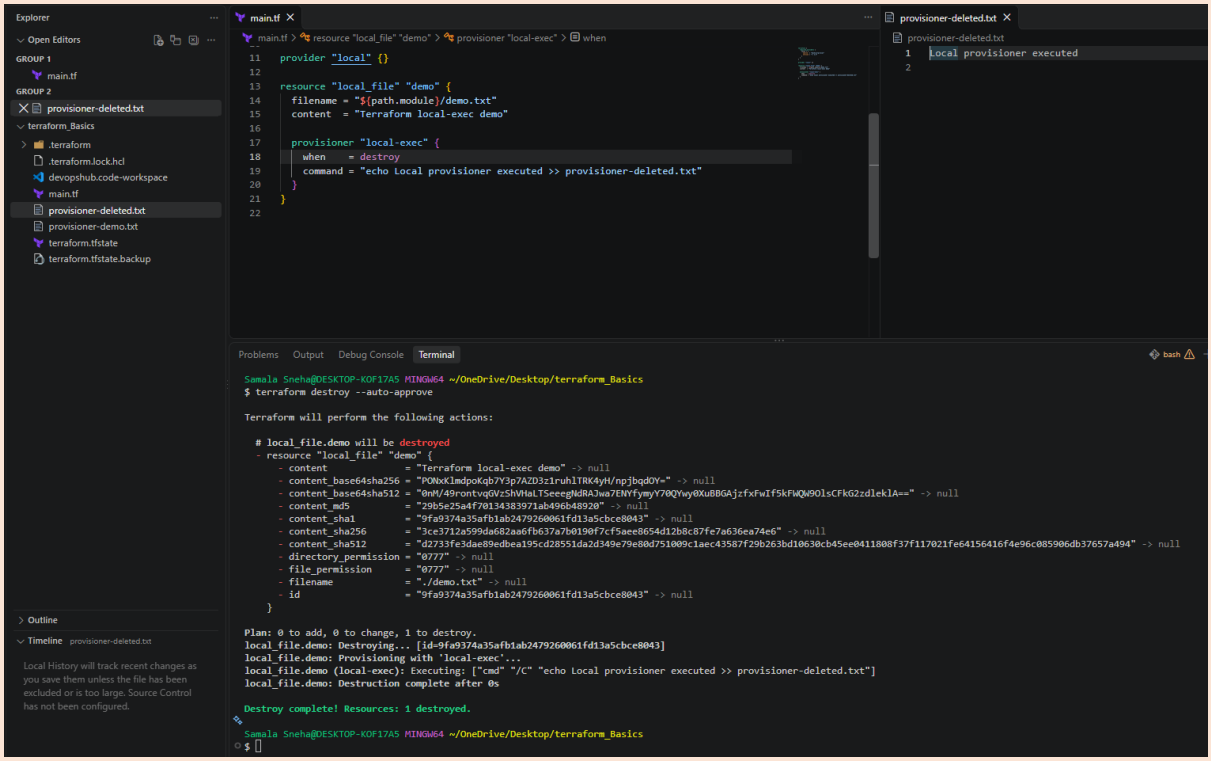
```
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply --auto-approve
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./demo.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
local_file.demo: Creating...
local_file.demo: Provisioning with 'local-exec'...
local_file.demo (local-exec): Executing: ["cmd" "/C" "exit 1"]

Error: local-exec provisioner error

  with local_file.demo,
  on main.tf line 17, in resource "local_file" "demo":
  17:   provisioner "local-exec" {

Error running command 'exit 1': exit status 1. Output:
```



11. Conclusion

In this lab, Terraform provisioners were used to execute commands locally and remotely. The local-exec provisioner demonstrated local command execution, while remote-exec demonstrated configuring an EC2 instance over SSH.

The lifecycle behaviors were also tested: default creation-time execution, destroy-time execution through when = destroy, and failure handling through on_failure = fail and on_failure = continue.

These concepts help DevOps engineers understand how Terraform interacts with resources beyond their initial creation. For production infrastructure, prefer cloud-init, EC2 user data, configuration-management tools, or purpose-built deployment workflows where appropriate.

Terraform Taint and Untaint

1. Objective

To understand how Terraform marks a resource for replacement using taint and how to remove that status.

2. Purpose

- **Taint:** Mark a resource as needing replacement during the next Terraform apply.
- **Untaint:** Remove the tainted status from a resource without replacing it.

Note: The terraform taint and terraform untaint commands are deprecated in modern Terraform. The recommended approach is terraform apply -replace="RESOURCE_ADDRESS" when you want to replace a resource.

3. Steps

Step 1: Check the resource address terraform state list

Step 2: Taint a resource (legacy command) terraform taint local_file.test

Terraform marks the resource for replacement during the next apply.

Step 3: Untaint the resource (legacy command) terraform untaint local_file.test

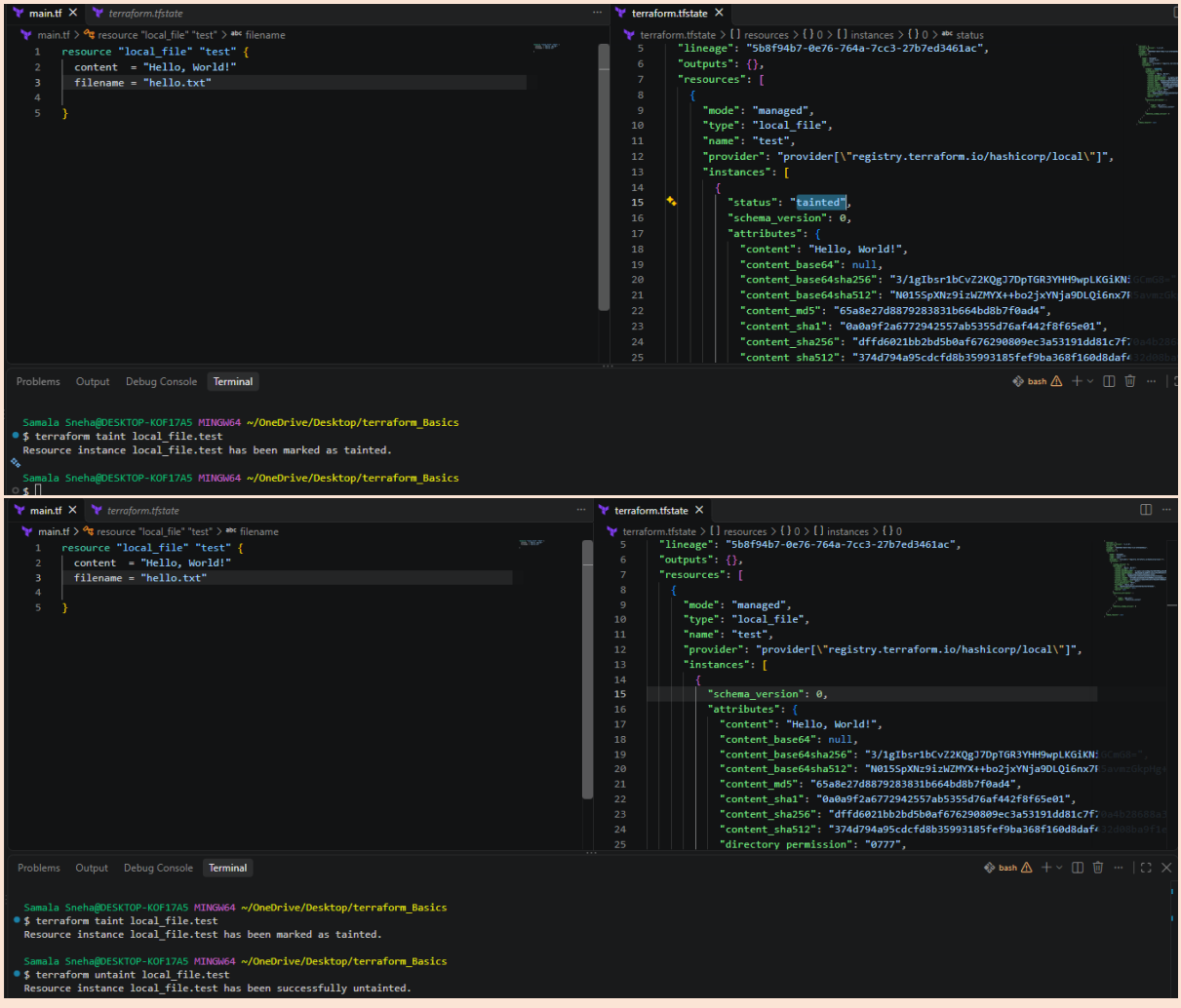
This removes the tainted status if the resource is still present in state.

Step 4: Recommended replacement method

To replace a resource directly using the modern approach:

→ terraform plan -replace="local_file.demo"
→ terraform apply -replace="local_file.demo"
Review the plan before confirming the apply.

4.Implementation



5. Conclusion

Tainting indicates that a resource should be replaced, while untainting removes that replacement status. For current Terraform workflows, use terraform apply -replace="RESOURCE_ADDRESS" when replacement is required.

Terraform Debugging — TRACE Logging

1. Objective

To enable Terraform TRACE logging, capture detailed debugging information, and disable logging when troubleshooting is complete.

2. Purpose

Terraform normally displays errors during commands such as terraform plan and terraform apply. When additional details are required, Terraform supports logging through the TF_LOG environment variable.

TRACE is the most detailed Terraform log level. It can expose extensive internal execution details, making it useful for deep troubleshooting.

3. Steps

Step 1: Enable TRACE logging in Git Bash

Run: export TF_LOG=TRACE

This enables the most detailed Terraform logs for the current terminal session.

Step 2: Save logs to a file

Run: export TF_LOG_PATH=/tmp/terraform.log

Terraform writes its logs to /tmp/terraform.log on the machine running Terraform.

Step 3: Save logs to a file permanently

Run: export TF_LOG_PATH=terraform.log
Terraform writes its logs to terraform.log on the machine running Terraform. Remove /tmp/

Step 4: Execute Terraform

Run a command that reproduces the issue: terraform plan Or terraform apply

Step 5: Inspect the log file

tail -n 50 /tmp/terraform.log

To search for errors: grep -iE "error|failed|timeout" /tmp/terraform.log

Step 6: Disable TRACE logging

Run:

→ unset TF_LOG
→ unset TF_LOG_PATH
This disables Terraform logging controlled by these variables in the current Git Bash session.

5. PowerShell Commands

If using the VS Code PowerShell terminal instead of Git Bash, use:

Enable TRACE logging: $env:TF_LOG = "TRACE"

Save logs to a file: $env:TF_LOG_PATH = "terraform.log"

Run Terraform: terraform plan

Disable logging:

→ Remove-Item Env:TF_LOG -ErrorAction SilentlyContinue
→ Remove-Item Env:TF_LOG_PATH -ErrorAction SilentlyContinue

6. Terraform Log Levels

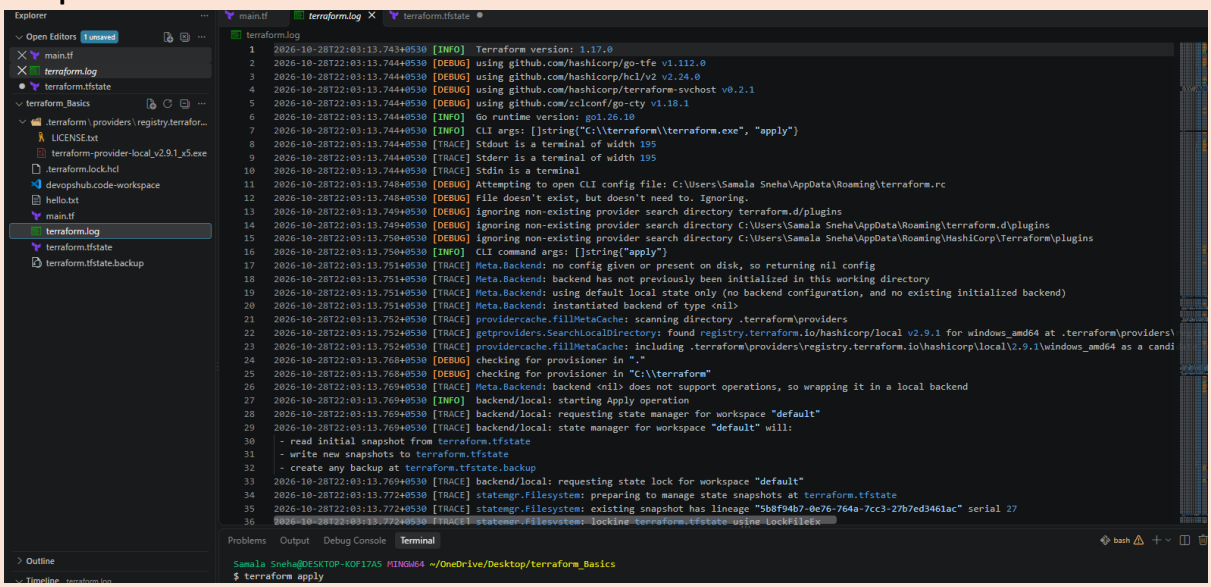
Terraform supports five TF_LOG levels, from least detailed to most detailed:

1. **INFO** — general informational messages.
2. **WARN** — warnings about potential issues.
3. **ERROR** — error messages.
4. **DEBUG** — detailed debugging information.
5. **TRACE** — the most detailed execution and internal diagnostic information.

Example: export TF_LOG=TRACE

Important: Terraform's supported levels are INFO, WARN, ERROR, DEBUG, and TRACE. The standard level is WARN, not WARNING.

7.Implementation



8. Conclusion

TRACE logging provides the deepest level of Terraform diagnostics and is useful when ordinary error messages are insufficient. Setting TF_LOG_PATH stores the logs for later analysis, while unsetting TF_LOG disables logging.

TRACE logs can contain sensitive infrastructure details. Store them securely and remove them when they are no longer required.

Terraform Import — Import Existing Infrastructure

1. Objective

To import an existing resource into Terraform state so Terraform can manage infrastructure that was created manually or outside the current Terraform configuration.

2. Purpose

The terraform import command associates an existing infrastructure resource with a Terraform resource address in the state file.

For example, if an S3 bucket already exists in AWS, Terraform can import it into state rather than attempting to create a new bucket with the same name.

Important: Importing a resource does not automatically generate its complete Terraform configuration. You must write the corresponding .tf resource block yourself.

4. Implementation Steps

Step 1: Create a project folder

Create a folder named: terraform-import-lab

Open this folder in VS Code and select the Git Bash terminal.

Step 2: Configure the AWS provider

Create a file named **main.tf**:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "existing" {
  bucket = "YOUR_EXISTING_BUCKET_NAME"
}
```

Replace YOUR_EXISTING_BUCKET_NAME with the name of an existing S3 bucket in your AWS account.

Step 3: Initialize Terraform

terraform init

This downloads the required provider.

Step 4: Import the existing S3 bucket

Run: terraform import aws_s3_bucket.existing YOUR_EXISTING_BUCKET_NAME

Replace the placeholder with the actual bucket name.

Example:

terraform import aws_s3_bucket.existing my-existing-bucket-12345

The example bucket name is illustrative. Import a real bucket that you own.

Step 5: Verify the imported resource

Run: terraform state list

Expected output: aws_s3_bucket.existing
local_file.test

Inspect the resource details: terraform state show aws_s3_bucket.existing

Terraform should display the attributes of the imported bucket.

Step 6: Check the configuration

Run: terraform plan

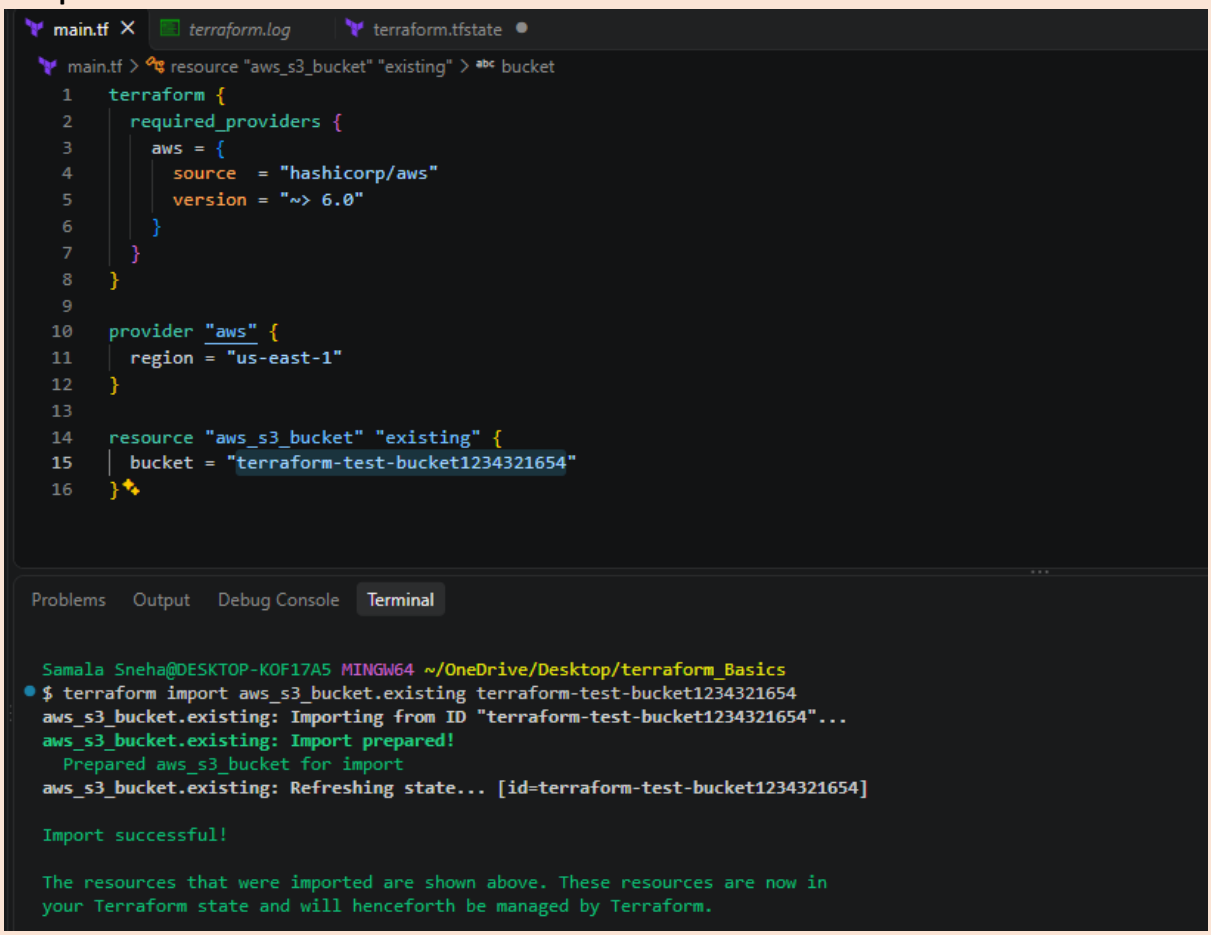
Review the plan carefully. If the configuration does not match the existing bucket's settings, Terraform may propose changes.

Adjust the Terraform configuration to match the existing resource before applying any changes.

5. Important Points

- terraform import imports an existing resource into Terraform state.
- The destination resource address must already be declared in the Terraform configuration.
- Importing does not create a new resource.
- Importing does not automatically generate the full .tf configuration.
- Import does not normally modify the remote resource itself, but a later terraform apply can change it if the configuration differs.
- Back up important state files and review plans before applying changes.

6.Implementation



7. Conclusion

Terraform import helps bring existing infrastructure under Terraform management without recreating it. By importing the resource, inspecting the state, and aligning the configuration with the real infrastructure, DevOps engineers can manage existing AWS resources through Infrastructure as Code.

TASK 2: CREATE AN EC2 INSTANCE WITH HTTPD USING TERRAFORM SCRIPT

1. Objective

To create an Amazon EC2 instance on AWS using Terraform, install Apache HTTP Server (httpd) automatically through a user-data script, and host a sample web page accessible through the instance's public IP address.

2. Purpose

The purpose of this task is to understand how Infrastructure as Code (IaC) simplifies AWS resource provisioning and server configuration.

Terraform is used to create and manage AWS resources through configuration files rather than manually creating them through the AWS Management Console.

This task demonstrates:

- Creating an EC2 instance using Terraform.
- Selecting an Amazon Linux 2023 AMI dynamically.
- Identifying the default VPC and an available subnet.
- Creating a security group to allow HTTP traffic.
- Installing Apache HTTP Server automatically using EC2 user data.
- Hosting a sample HTML web page.
- Retrieving EC2 details using Terraform output values.
- Verifying the deployed website through a browser.

3. Prerequisites

The following tools and configurations are required:

1. Visual Studio Code (VS Code).
2. Terraform installed on Windows.
3. AWS CLI installed and configured.
4. Git Bash terminal.
5. An AWS account with the required EC2, VPC, security group, and related IAM permissions.
6. AWS region configured as us-east-1 (N. Virginia).
7. A default VPC with at least one subnet and internet connectivity for the EC2 instance.

Verify the prerequisites

Execute the following commands in the VS Code Git Bash terminal.

→ terraform -version
→ aws --version
→ aws sts get-caller-identity
The commands verify the Terraform installation, AWS CLI installation, and AWS authentication.

4. Project Structure

The Terraform project was created in the following directory: terraform-ec2-httpd
The project contains these files:

```
terraform-ec2-httpd/
 ├── main.tf
 ├── outputs.tf
 ├── .terraform/
 ├── .terraform.lock.hcl
 └── terraform.tfstate
```

The .terraform directory, dependency lock file, and state file are generated or maintained by Terraform during initialization and execution. The state file is local at this stage and will be migrated to an S3 backend in Task 4.

5. Implementation Steps

Step 1: Create the project directory

Open VS Code and select Terminal → New Terminal.

Execute the following commands in Git Bash: created 1 file in desktop and opened Vs Code and opened that folder in vs code and created the files
If the project directory already exists, navigate into it rather than creating it again.

Create two Terraform files:

- main.tf
- outputs.tf

Step 2: Configure the AWS provider and resources

The **main.tf** file contains the Terraform configuration used to create the infrastructure.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Find the latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
```

```
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Find the default VPC
data "aws_vpc" "default" {
  default = true
}

# Find subnets in the default VPC
data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# Allow HTTP traffic
resource "aws_security_group" "httpd_sg" {
  name        = "terraform-httpd-sg"
  description = "Allow HTTP access"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }
}

# Create EC2 and install Apache HTTP Server
resource "aws_instance" "web" {
  ami                    = data.aws_ami.amazon_linux.id
  instance_type          = "t3.micro"
  subnet_id              = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.httpd_sg.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y httpd
    systemctl enable --now httpd

    cat > /var/www/html/index.html <<'HTML'
    <html>
      <body>
        <h1>Welcome to Terraform EC2</h1>
        <p>Apache HTTP Server is running successfully!</p>
      </body>
    </html>
    HTML
  EOF

  tags = {
    Name = "Terraform-HTTPD-Server"
  }
}
```

Save the file using Ctrl + S.

Step 3: Understand the configuration

Terraform and provider block: Specifies the AWS provider and the target region.

AMI data source: Searches for the latest matching Amazon Linux 2023 x86-64 AMI published by Amazon.

VPC and subnet data sources: Discover the default VPC and its available subnets.

Security group: Allows inbound TCP traffic on port 80 so users can access the website over HTTP. Outbound traffic is permitted.

EC2 instance: Creates a t3.micro instance using the selected AMI and subnet.

User data: Executes a shell script during the instance's initial boot. It installs Apache, starts the service, enables it at boot, and writes a sample HTML page to the Apache document root.

Tags: Assigns the name Terraform-HTTPD-Server to the EC2 instance.

Step 4: Configure Terraform outputs

Create **outputs.tf** and add the following code:

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP address of the web server"
  value       = aws_instance.web.public_ip
}

output "website_url" {
  description = "URL of the Apache web server"
  value       = "http://${aws_instance.web.public_ip}"
}
```

Save the file.

These output blocks display the instance ID, public IP address, and website URL after Terraform completes the deployment.

Step 5: Initialize Terraform

Execute: terraform init

This command initializes the Terraform working directory and downloads the required AWS provider.

Step 6: Format and validate the configuration

Execute:

→ terraform fmt
→ terraform validate

terraform fmt formats the configuration files according to Terraform's standard formatting rules.
terraform validate checks the configuration for syntax errors and structural issues.

Step 7: Review the execution plan

Execute: terraform plan

Terraform evaluates the configuration and displays the resources it intends to create.
Review the proposed changes before proceeding.

Step 8: Create the infrastructure

Execute: terraform apply

Review the execution plan and enter: yes

Terraform creates the security group and EC2 instance. During the instance's initial boot, the user-data script installs and starts Apache HTTP Server.

Step 9: Retrieve the Terraform outputs

Execute: terraform output

To retrieve individual values:

→ terraform output -raw instance_id
→ terraform output -raw public_ip
→ terraform output -raw website_url

6. Verification and Results

The following results were observed after successful deployment.

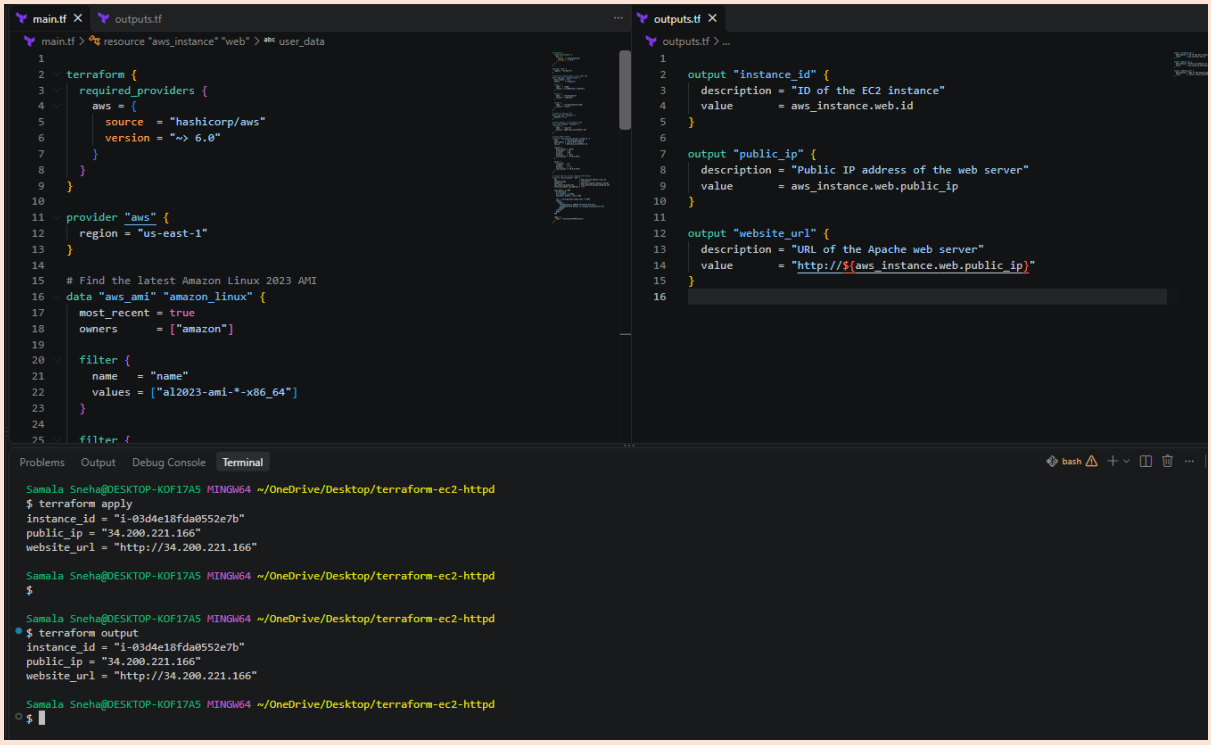
EC2 instance ID: i-03d4e18fda0552e7b

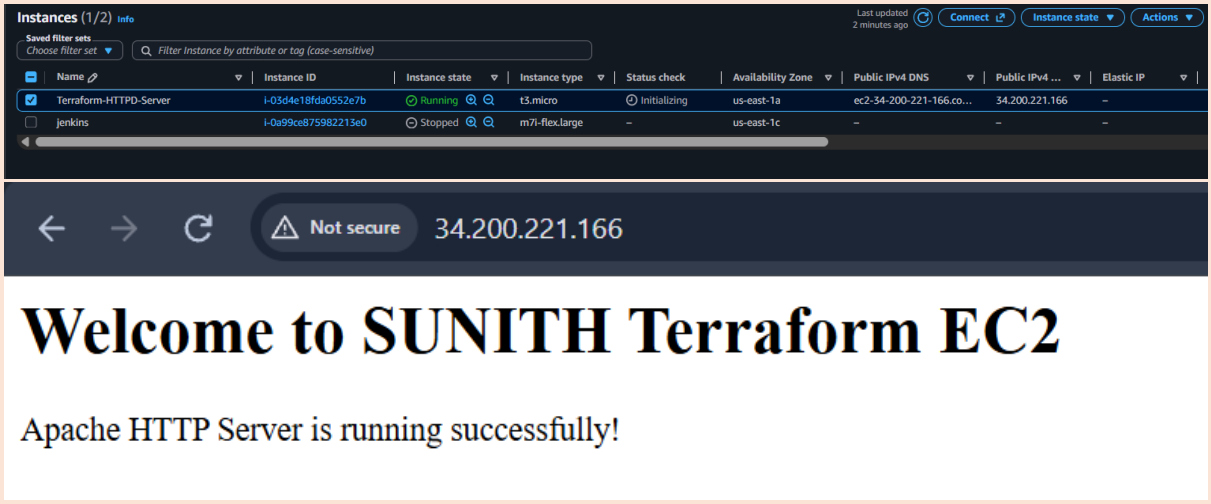
Public IP address: 34.200.221.166

Website URL: http://34.200.221.166

The website was opened successfully in the browser, confirming that the EC2 instance was reachable over HTTP and the Apache-hosted sample page was being served.

7. Implementation





8. Conclusion

In this task, an Amazon EC2 instance was successfully provisioned using Terraform. The configuration dynamically discovered an Amazon Linux 2023 AMI, selected a subnet in the default VPC, created a security group, and installed Apache HTTP Server through EC2 user data.

Terraform outputs were used to retrieve the instance ID, public IP address, and website URL. The web page was successfully accessed through a browser, confirming that the deployment worked as intended.

This task demonstrates the practical use of Infrastructure as Code to automate AWS infrastructure provisioning and basic web-server configuration.

TASK 3: CONFIGURE AMAZON S3 AS THE TERRAFORM REMOTE BACKEND TASK 3

1. Objective

To configure Amazon S3 as the remote backend for Terraform and migrate the existing local Terraform state file to an S3 bucket without recreating the existing EC2 instance.

2. Purpose

The purpose of this task is to understand remote state management in Terraform.

Terraform normally stores its state in a local file named terraform.tfstate. Storing the state in Amazon S3 allows authorized users and automation systems to access the same remote state, supporting team collaboration and centralized infrastructure management.

This task demonstrates:

- Creating an S3 bucket using Terraform.
- Enabling S3 bucket versioning.
- Enabling server-side encryption.
- Blocking public access to the state bucket.
- Configuring the S3 backend.
- Migrating existing Terraform state from local storage to S3.
- Verifying that the existing EC2 instance remains managed by Terraform.

3. Prerequisites

The following prerequisites are required:

1. VS Code installed on Windows.
2. Git Bash terminal.
3. Terraform and AWS CLI installed.
4. AWS credentials configured with the required permissions.
5. Task 3 completed successfully.
6. Existing Terraform configuration and local state available.
7. AWS region set to us-east-1.

Verify the AWS identity and Terraform installation: aws sts get-caller-identity

→ terraform -version

4. Existing Project Structure

The existing project directory was: ~/OneDrive/Desktop/terraform-ec2-httpd

The project contained:

- main.tf — AWS provider and infrastructure configuration.
- outputs.tf — Terraform output definitions.
- terraform.tfstate — Local Terraform state before migration.

The existing EC2 instance and security group were retained during the backend configuration process.

5. Implementation Steps

Step 1: Add the S3 bucket configuration

The following configuration was added to the existing main.tf file.

```
data "aws_caller_identity" "current" {}

resource "aws_s3_bucket" "terraform_state" {
  bucket = "terraform-httpd-state-${data.aws_caller_identity.current.account_id}-us-east-1"

  tags = {
    Name    = "Terraform-State-Bucket"
    Purpose = "Terraform Remote State"
  }
}

resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

The bucket name incorporates the AWS account ID to reduce the likelihood of a naming conflict.

The configuration enables versioning, server-side encryption using AES-256, and all four S3 public-access-block settings.

Step 2: Add the bucket output

The following block was added to outputs.tf.

```
output "terraform_state_bucket" {
  description = "S3 bucket used for Terraform state"
  value       = aws_s3_bucket.terraform_state.bucket
}
```

Step 3: Validate and create the bucket

The following commands were executed in the VS Code Git Bash terminal.

```
→ terraform fmt
→ terraform validate
→ terraform plan
→ terraform apply
```

The terraform apply command was confirmed after reviewing the proposed changes.

Terraform created the S3 bucket and its associated configuration resources.

Step 4: Configure the S3 backend

The following backend configuration was added inside the existing terraform block in main.tf.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket  = "terraform-httpd-state-567487919764-us-east-1"
    key     = "terraform-ec2-httpd/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}
```

The bucket name was taken from the actual Terraform output.

The key identifies the location of the Terraform state object inside the bucket. The region specifies the AWS region, and encrypt = true enables encryption for the backend's state storage.

Step 5: Migrate the existing state

The following command was executed: `terraform init -migrate-state`

Terraform initialized the S3 backend and migrated the existing local state to S3.

This migration preserved the relationship between the Terraform configuration and the already-created EC2 instance.

Step 6: Verify the Terraform state

The following command was executed: terraform state list

The resulting resource list included:

- data.aws_ami.amazon_linux
- data.aws_caller_identity.current
- data.aws_subnets.default
- data.aws_vpc.default
- aws_instance.web
- aws_s3_bucket.terraform_state
- aws_s3_bucket_public_access_block.terraform_state
- aws_s3_bucket_server_side_encryption_configuration.terraform_state
- aws_s3_bucket_versioning.terraform_state
- aws_security_group.httpd_sg

This confirmed that the EC2 instance, security group, and S3 bucket resources remained tracked by Terraform.

Step 7: Verify Terraform outputs

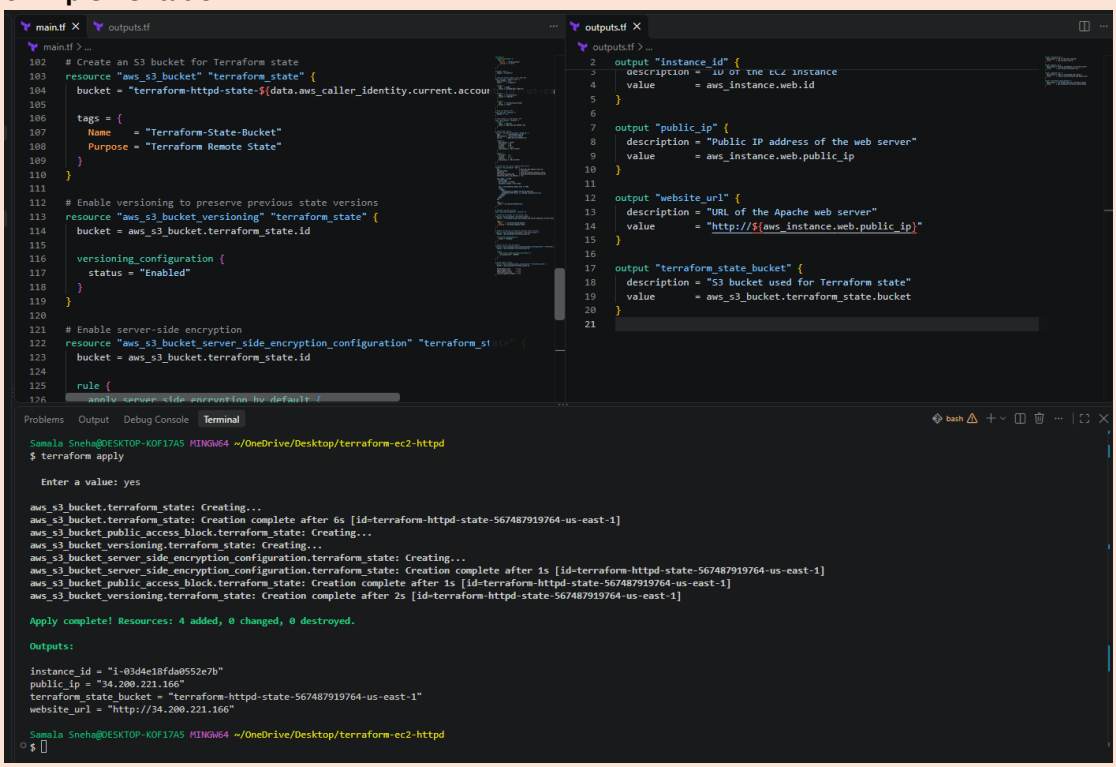
The following command was executed: terraform output

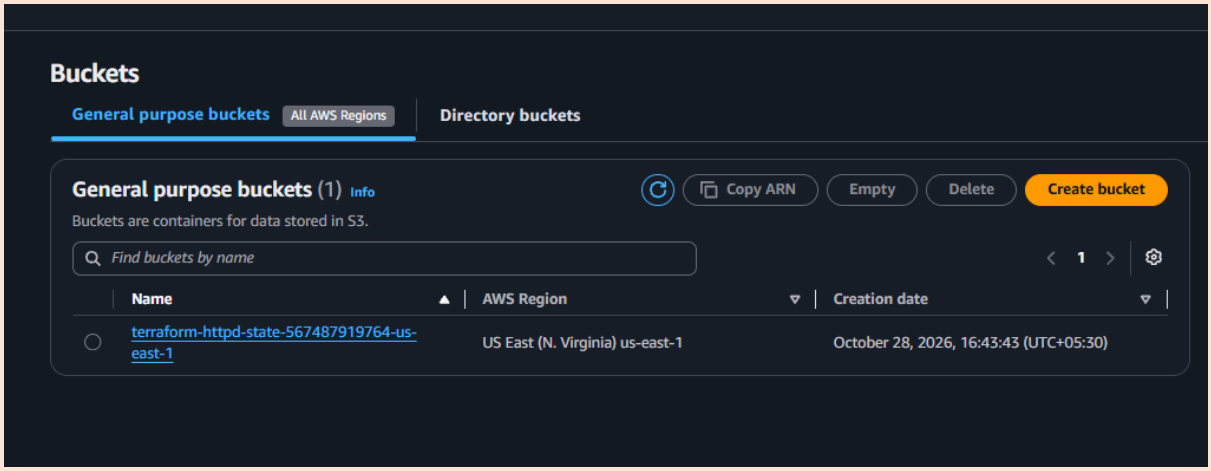
The verified output was:

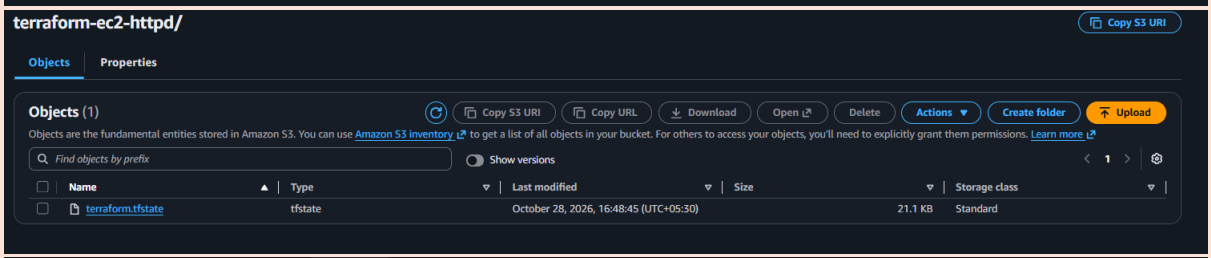
- instance_id = "i-03d4e18fda0552e7b"
- public_ip = "34.200.221.166"
- terraform_state_bucket = "terraform-httpd-state-567487919764-us-east-1"
- website_url = "http://34.200.221.166"

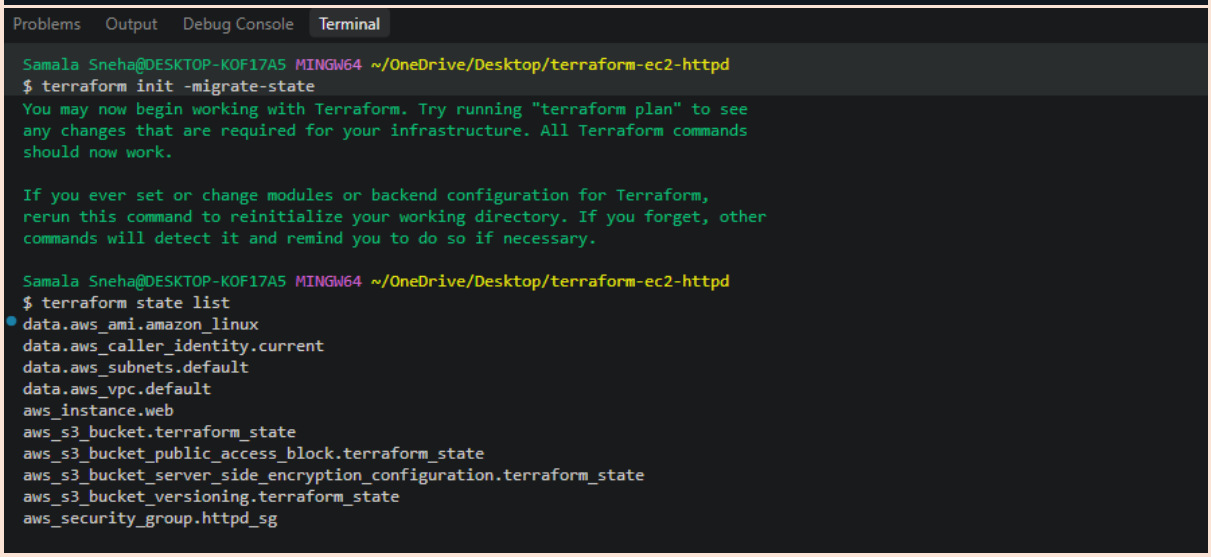
The S3 state object was also visually confirmed in the AWS Management Console.

6. Implementation









7. Conclusion

Amazon S3 was successfully configured as the Terraform remote backend. The S3 bucket was created with versioning, encryption, and public-access blocking enabled. The existing local state was migrated to the remote backend, and the existing EC2 instance continued to be tracked by Terraform.

This task demonstrated centralized Terraform state storage, an important practice for managing infrastructure consistently across development environments and teams.

TASK 4: CONFIGURE DYNAMODB STATE LOCKING FOR TERRAFORM FOR TASK 3

1. Objective

To configure Amazon DynamoDB for Terraform state locking in an existing Terraform project that uses Amazon S3 as its remote backend.

2. Purpose

The purpose of this task is to understand how Terraform state locking helps prevent simultaneous operations from modifying the same Terraform state.

When multiple users or automation processes work with the same Terraform state, concurrent operations can cause conflicts. State locking coordinates operations so that only an authorized operation holds the lock at a time.

This task demonstrates:

- Creating an Amazon DynamoDB table using Terraform.
- Configuring the required LockID partition key.
- Connecting DynamoDB locking to an existing S3 backend.
- Reinitializing Terraform after a backend configuration change.
- Verifying the DynamoDB table.
- Confirming that Terraform can acquire and release the state lock.
- Verifying that the existing EC2 instance and other AWS resources remain managed by Terraform.

3. Prerequisites

The following prerequisites are required:

1. VS Code installed on Windows.
2. Git Bash terminal.
3. Terraform and AWS CLI installed.
4. AWS credentials configured with the required permissions.
5. Task 3 completed: EC2 instance with Apache HTTP Server.
6. Task 4 completed: S3 remote backend configured and state migrated.
7. An S3 bucket containing the Terraform state.
8. Permission to create and manage DynamoDB tables.

The existing project directory is: ~/OneDrive/Desktop/terraform-ec2-httpd

The AWS region is us-east-1 (N. Virginia).

4. Existing Infrastructure

Before implementing DynamoDB locking, the project already contained:

- An EC2 instance running Apache HTTP Server.
- A security group allowing inbound HTTP traffic.
- An S3 bucket configured for Terraform remote state.
- An S3 backend configured in main.tf.

The existing S3 bucket is: terraform-httpd-state-567487919764-us-east-1

The Terraform state object is stored at: terraform-ec2-httpd/terraform.tfstate

5. Implementation Steps

Step 1: Create the DynamoDB table

The following resource was added to the existing **main.tf** file.

```
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "terraform-httpd-state-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name    = "Terraform-State-Locks"
    Purpose = "Terraform State Locking"
  }
}
```

Explanation:

- aws_dynamodb_table creates a DynamoDB table.
- name specifies the table name.
- billing_mode = "PAY_PER_REQUEST" uses on-demand billing.
- hash_key = "LockID" defines the partition key.
- type = "S" specifies that the key is a string.
- tags identify the table's purpose.

The S3 backend's DynamoDB locking integration requires a string partition key named LockID.

Step 2: Add the DynamoDB output

The following block was added to the existing **outputs.tf** file.

```
output "terraform_lock_table" {
  description = "DynamoDB table used for Terraform state locking"
  value       = aws_dynamodb_table.terraform_locks.name
}
```

This output allows the table name to be retrieved directly from Terraform.

Step 3: Format, validate, and review the configuration

The following commands were executed in the VS Code Git Bash terminal.

→ terraform fmt
→ terraform validate
→ terraform plan

These commands format the Terraform files, validate the configuration, and preview the proposed infrastructure changes.

Step 4: Create the DynamoDB table

The following command was executed: terraform apply

After reviewing the execution plan, the operation was confirmed.

Terraform created the DynamoDB table and began managing it through the existing S3-backed state.

Step 5: Configure DynamoDB locking in the S3 backend

The following configuration was added to the existing backend "s3" block in **main.tf**.

```
backend "s3" {
  bucket       = "terraform-httpd-state-567487919764-us-east-1"
  key          = "terraform-ec2-httpd/terraform.tfstate"
  region       = "us-east-1"
  encrypt      = true
  dynamodb_table = "terraform-httpd-state-locks"
}
```

The dynamodb_table setting identifies the DynamoDB table used for state locking.

The bucket and key continue to identify the same remote state location.

Note: Terraform currently marks dynamodb_table as deprecated and recommends S3-native locking through use_lockfile. The DynamoDB setting was retained here to meet the assignment requirement.

Step 6: Reinitialize Terraform

The following command was executed: terraform init -reconfigure

Terraform reported that the S3 backend was configured successfully.

It also displayed a deprecation warning for dynamodb_table. Despite the warning, initialization completed successfully.

Step 7: Verify the DynamoDB table

The following AWS CLI command was executed:

```
aws dynamodb describe-table \
  --table-name terraform-httpd-state-locks \
  --region us-east-1 \
  --query "Table.{Name:TableName,Status:TableStatus,Key:KeySchema}"
```

The actual output confirmed:

```
{
  "Name": "terraform-httpd-state-locks",
  "Status": "ACTIVE",
  "Key": [
    {
      "AttributeName": "LockID",
      "KeyType": "HASH"
    }
  ]
}
```

This confirmed that the DynamoDB table was active and had the required partition key.

Step 8: Verify Terraform state resources

The following command was executed: terraform state list

The returned resource list included:

- data.aws_ami.amazon_linux
- data.aws_caller_identity.current
- data.aws_subnets.default
- data.aws_vpc.default
- aws_dynamodb_table.terraform_locks
- aws_instance.web
- aws_s3_bucket.terraform_state
- aws_s3_bucket_public_access_block.terraform_state
- aws_s3_bucket_server_side_encryption_configuration.terraform_state
- aws_s3_bucket_versioning.terraform_state
- aws_security_group.httpd_sg

This confirmed that the DynamoDB table, EC2 instance, S3 bucket, and security group remained tracked by Terraform.

Step 9: Verify Terraform outputs

The following command was executed: terraform output

The verified output was:

- instance_id = "i-03d4e18fda0552e7b"
- public_ip = "34.200.221.166"
- terraform_lock_table = "terraform-httpd-state-locks"
- terraform_state_bucket = "terraform-httpd-state-567487919764-us-east-1"
- website_url = "http://34.200.221.166"

These outputs confirmed that the original EC2 instance and the backend resources were still available.

Step 10: Verify state locking

The following command was executed: terraform plan

The terminal displayed: Acquiring state lock. This may take a few moments...

Terraform then refreshed the managed resources and reported:
No changes. Your infrastructure matches the configuration.

Finally, the terminal displayed: Releasing state lock. This may take a few moments...

This confirmed that Terraform acquired and released a state lock during the plan operation.

6. Results

The task was completed successfully.

Component	Verified result
DynamoDB table	terraform-httpd-state-locks

Table status	ACTIVE
Partition key	LockID
S3 backend	Configured successfully
Terraform state	Stored in S3
EC2 instance	i-03d4e18fda0552e7b
Public IP	34.200.221.166
Terraform plan	No changes
Lock operation	Acquired and released successfully

7. Important Note About Deprecation

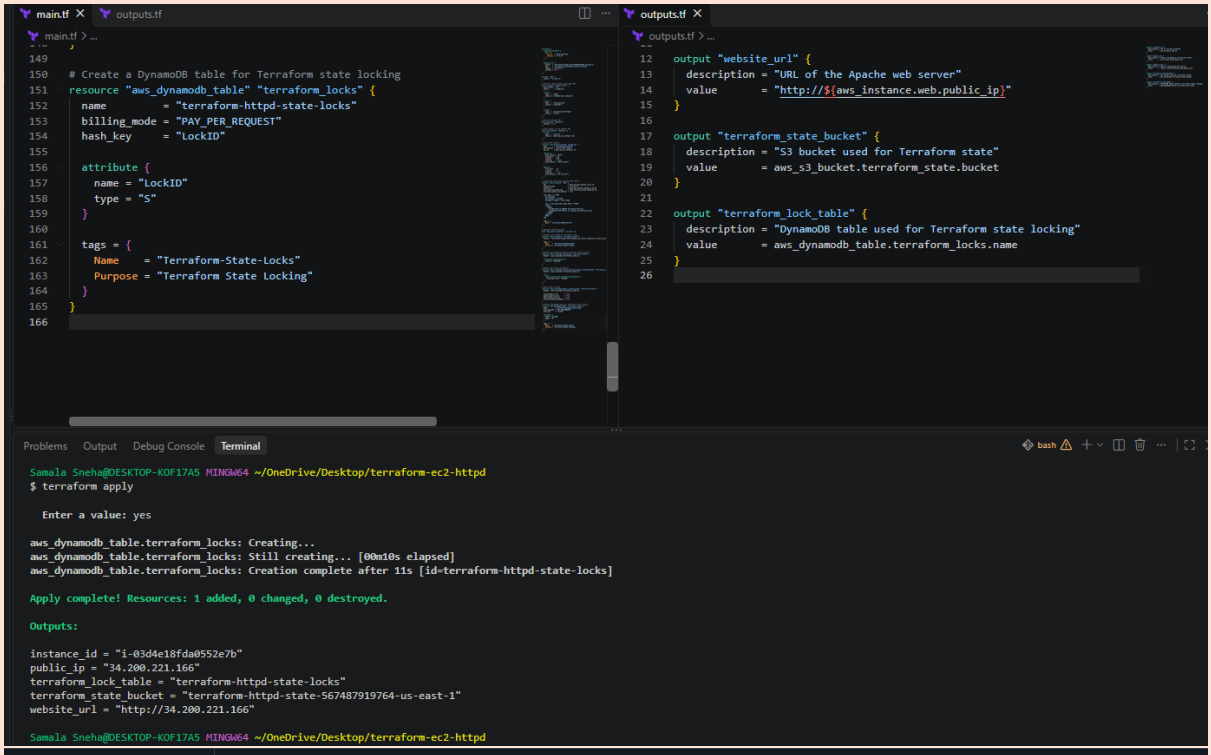
Terraform v1.17.0 displayed the following warning:

Warning: Deprecated Parameter

The parameter "dynamodb_table" is deprecated.
Use parameter "use_lockfile" instead.
This warning does not mean that the configuration failed. The initialization and plan operations succeeded, and Terraform acquired and released a state lock.

For new projects, S3-native locking is the recommended approach. The DynamoDB configuration is retained in this exercise to demonstrate the requested DynamoDB locking mechanism.

8.Implementation



```
# Create a DynamoDB table for Terraform state locking
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "terraform-httpd-state-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name    = "Terraform-State-Locks"
    Purpose = "Terraform State Locking"
  }
}
```

```
output "website_url" {
  description = "URL of the Apache web server"
  value       = "http://${aws_instance.web.public_ip}"
}

output "terraform_state_bucket" {
  description = "S3 bucket used for Terraform state"
  value       = aws_s3_bucket.terraform_state.bucket
}

output "terraform_lock_table" {
  description = "DynamoDB table used for Terraform state locking"
  value       = aws_dynamodb_table.terraform_locks.name
}
```

```
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform-ec2-httpd
$ terraform apply

  Enter a value: yes

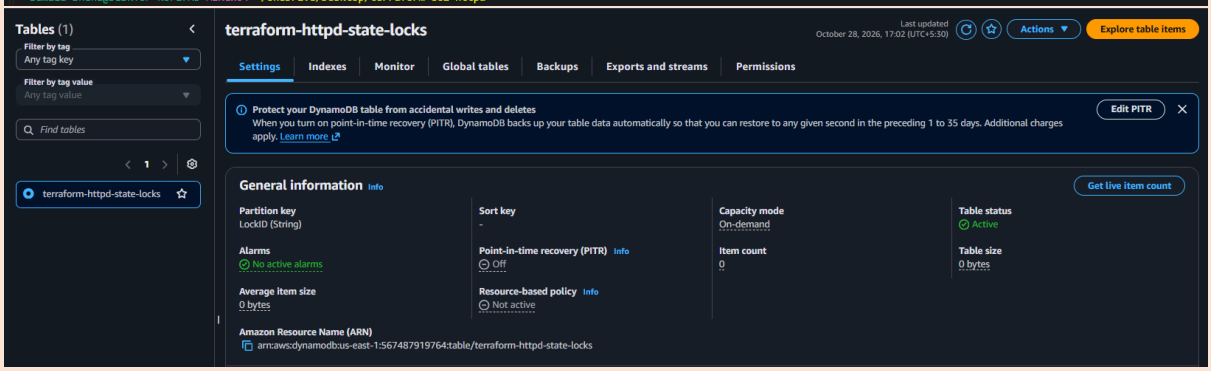
aws_dynamodb_table.terraform_locks: Creating...
aws_dynamodb_table.terraform_locks: Still creating... [00m10s elapsed]
aws_dynamodb_table.terraform_locks: Creation complete after 11s [id=terraform-httpd-state-locks]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-03d4e18fda0552e7b"
public_ip = "34.200.221.166"
terraform_lock_table = "terraform-httpd-state-locks"
terraform_state_bucket = "terraform-httpd-state-567487919764-us-east-1"
website_url = "http://34.200.221.166"

Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform-ec2-httpd
```

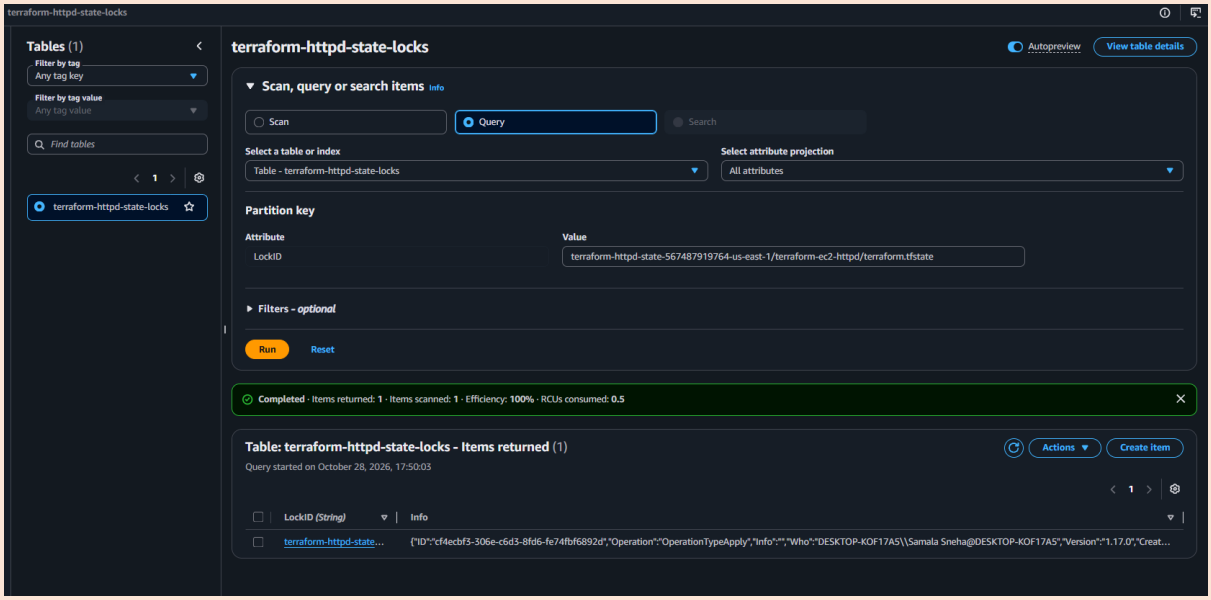



```
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform-ec2-httpd
$ terraform plan

  The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

Acquiring state lock. This may take a few moments...
data.aws_vpc.default: Reading...
data.aws_caller_identity.current: Reading...
data.aws_ami.amazon_linux: Reading...
aws_dynamodb_table.terraform_locks: Refreshing state... [id=terraform-httpd-state-locks]
data.aws_caller_identity.current: Read complete after 0s [id=567487919764]
aws_s3_bucket.terraform_state: Refreshing state... [id=terraform-httpd-state-567487919764-us-east-1]
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-0311d59d4887ec03f]
data.aws_vpc.default: Read complete after 2s [id=vpc-0fff3eb0730a112f0]
data.aws_subnets.default: Reading...
aws_security_group.httpd_sg: Refreshing state... [id=sg-0678f69edec7e210b]
data.aws_subnets.default: Read complete after 1s [id=us-east-1]
aws_instance.web: Refreshing state... [id=i-03d4e18fda0552e7b]
aws_s3_bucket_versioning.terraform_state: Refreshing state... [id=terraform-httpd-state-567487919764-us-east-1]
aws_s3_bucket_public_access_block.terraform_state: Refreshing state... [id=terraform-httpd-state-567487919764-us-east-1]
aws_s3_bucket_server_side_encryption_configuration.terraform_state: Refreshing state... [id=terraform-httpd-state-567487919764-us-east-1]

No changes. Your infrastructure matches the configuration.
```



9. Conclusion

Amazon DynamoDB was successfully configured for Terraform state locking with the existing S3 remote backend. The table was verified as active, and its required LockID partition key was confirmed.

Terraform successfully initialized the backend, acquired and released a state lock during the planning operation, and reported that the infrastructure matched the configuration. The existing EC2 instance and other resources continued to be tracked without requiring recreation.

This task demonstrated the role of state locking in coordinating Terraform operations and protecting shared infrastructure state from concurrent modifications.

The three tasks are now complete:

1. EC2 instance creation and Apache HTTP Server installation using Terraform.
2. S3 remote backend configuration and state migration.
3. DynamoDB state locking configuration and verification.

TASK 5: IMPLEMENT AND VERIFY TERRAFORM STATE LOCKING USING AMAZON DYNAMODB

1. Objective

To implement and verify Terraform state locking using an Amazon DynamoDB table integrated with an Amazon S3 remote backend. The purpose is to prevent multiple Terraform operations from modifying the same state simultaneously.

2. Purpose

Terraform maintains a state file that records the resources it manages. When multiple users or processes attempt to modify the same state at the same time, conflicts may occur.

To avoid this problem, Terraform supports state locking. In this lab:

- Amazon S3 stores the remote Terraform state file.
- Amazon DynamoDB provides the locking mechanism.
- Terraform automatically acquires a lock before an operation and releases it after completion.
- The temporary lock record can be inspected in DynamoDB while the operation is running.

3. Prerequisites

1. An AWS account with access to Amazon S3, DynamoDB and the required EC2 resources.
2. AWS CLI installed and configured on Windows.
3. Terraform installed and accessible from Git Bash.
4. A working Terraform project.
5. An existing S3 bucket containing the remote Terraform state.
6. An existing DynamoDB table with the following configuration:

Property	Value
Table name	terraform-httpd-state-locks
AWS Region	us-east-1
Partition key	LockID
Partition key type	String
Billing mode	On-demand
Table status	Active

4. Terraform Backend Configuration

The S3 backend is configured in main.tf as follows:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket       = "terraform-httpd-state-567487919764-us-east-1"
    key          = "terraform-ec2-httpd/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    dynamodb_table = "terraform-httpd-state-locks"
  }
}
```

Explanation

- bucket: Specifies the S3 bucket that stores the Terraform state.
- key: Specifies the location of the state object inside the bucket.
- region: Specifies the AWS region.
- encrypt: Enables encryption for the remote state object.
- dynamodb_table: Specifies the DynamoDB table used for state locking.

Note: The dynamodb_table backend parameter is deprecated in current Terraform documentation. This lab retains it to demonstrate the requested DynamoDB-based locking mechanism.

5. Implementation Steps

Step 1: Verify AWS CLI access

Open Git Bash and navigate to the Terraform project directory:
`cd ~/OneDrive/Desktop/terraform-ec2-httpd`

Verify the AWS account: `aws sts get-caller-identity`

Verify the DynamoDB table:

```
aws dynamodb describe-table \
  --table-name terraform-httpd-state-locks \
  --region us-east-1 \
  --query "Table.{Name:TableName,Status:TableStatus,Key:KeySchema}"
```

Expected result: the table status should be ACTIVE, and the partition key should be LockID of type String.

2: Verify Terraform initialization

Run: terraform init

Terraform should initialize the S3 backend and load the required provider configuration.

If the backend configuration has changed, Terraform may require backend reconfiguration or state migration. Follow the appropriate instructions for the change rather than running migration commands unnecessarily.

Step 3: Monitor the DynamoDB lock record

Run the following command in the first Git Bash terminal:

```
while true; do
  aws dynamodb scan \
    --table-name terraform-httpd-state-locks \
    --region us-east-1 \
    --consistent-read \
    --query "Items[*].LockID.S" \
    --output text
  sleep 0.2
done
```

This command repeatedly scans the DynamoDB table every 0.2 seconds and displays any available lock IDs.

Keep this terminal running.

Step 4: Execute Terraform in a second terminal

Open another Git Bash terminal in the same project directory and execute: terraform apply

Terraform refreshes the resources, checks the configuration and determines whether changes are necessary.

During the operation, the first terminal may display a lock ID similar to:
terraform-httpd-state-567487919764-us-east-1/terraform-ec2-httpd/terraform.tfstate

This value identifies the state file associated with the lock.

Step 5: Verify the lock in the AWS Console

1. Sign in to the AWS Management Console.
2. Open Amazon DynamoDB.
3. Select the terraform-httpd-state-locks table.
4. Open **Explore items**.
5. Select **Scan** and click **Run** while the lock record is present.
6. Inspect the returned LockID attribute.

The lock record is temporary. If Terraform has already completed, the record may no longer exist.

Step 6: Understand the Query operation

The Query operation requires the exact partition-key value.

To query a lock record:

1. Select **Query** in Explore items.
2. Set the partition-key attribute to LockID.
3. Enter the exact lock ID captured from the monitoring terminal.
4. Click **Run** while the record still exists.

If the lock has already been released, the query returns zero items. This does not by itself indicate a failure.

Step 7: Confirm successful completion

The expected Terraform output includes:

Acquiring state lock. This may take a few moments...

No changes. Your infrastructure matches the configuration.

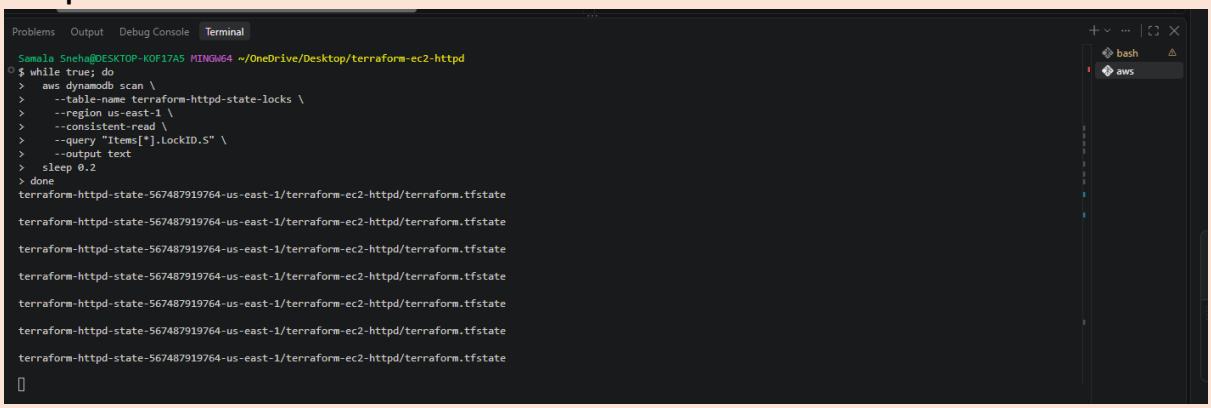
Releasing state lock. This may take a few moments...

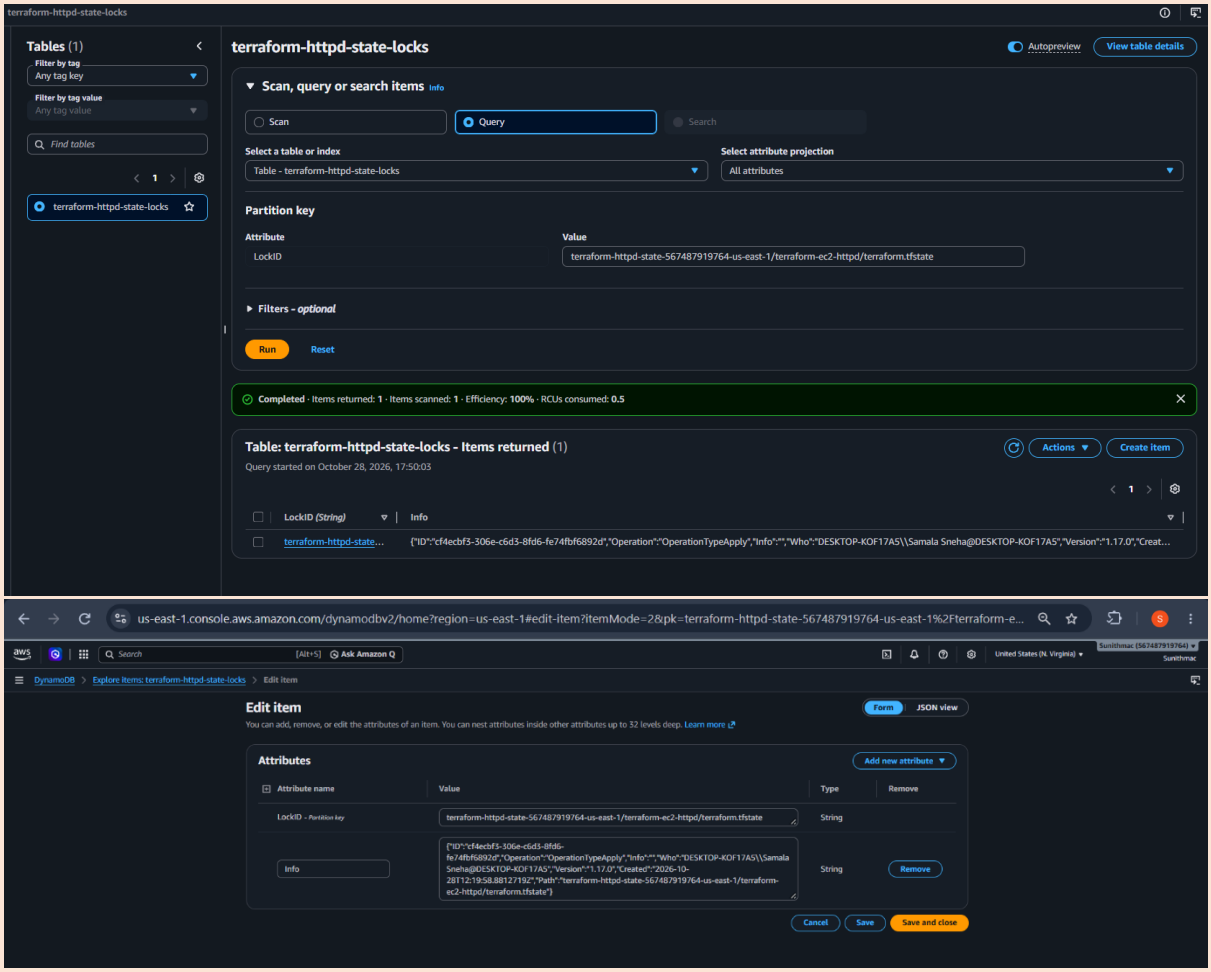
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

These messages, together with the observed DynamoDB lock record, demonstrate the locking workflow.

Stop the monitoring loop using Ctrl+C after verification.

6. Implementation





7. Conclusion

Successfully implemented and verified Terraform state locking using an Amazon DynamoDB table and an Amazon S3 remote backend. The lock record was observed during Terraform execution, and Terraform completed the operation and released the lock automatically.

This configuration demonstrates how state locking helps prevent concurrent modifications to the same Terraform state file.

Final Complete Code (for Task 3,4,5) → Complete Main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket         = "terraform-httpd-state-567487919764-us-east-1"
    key            = "terraform-ec2-httpd/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-httpd-state-locks"
  }

}

provider "aws" {
  region = "us-east-1"
}

# Find the latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Find the default VPC
data "aws_vpc" "default" {
```

```
  default = true
}

# Find subnets in the default VPC
data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# Allow HTTP traffic
resource "aws_security_group" "httpd_sg" {
  name        = "terraform-httpd-sg"
  description = "Allow HTTP access"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Create EC2 and install Apache HTTP Server
resource "aws_instance" "web" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = "t3.micro"
  subnet_id                   = data.aws_subnets.default.ids[0]
  vpc_security_group_ids      = [aws_security_group.httpd_sg.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y httpd
    systemctl enable --now httpd

    cat > /var/www/html/index.html <<'HTML'
    <html>
```

```
    <body>
      <h1>Welcome to SUNITH Terraform EC2</h1>
      <p>Apache HTTP Server is running successfully!</p>
    </body>
   </html>
   HTML
 EOF

 tags = {
   Name = "Terraform-HTTPD-Server"
 }
}

# Identify the AWS account
data "aws_caller_identity" "current" {}

# Create an S3 bucket for Terraform state
resource "aws_s3_bucket" "terraform_state" {
 bucket = "terraform-httpd-state-${data.aws_caller_identity.current.account_id}-us-east-1"

 tags = {
   Name    = "Terraform-State-Bucket"
   Purpose = "Terraform Remote State"
 }
}

# Enable versioning to preserve previous state versions
resource "aws_s3_bucket_versioning" "terraform_state" {
 bucket = aws_s3_bucket.terraform_state.id

 versioning_configuration {
   status = "Enabled"
 }
}

# Enable server-side encryption
resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
 bucket = aws_s3_bucket.terraform_state.id

 rule {
   apply_server_side_encryption_by_default {
     sse_algorithm = "AES256"
   }
 }
}

# Block public access
resource "aws_s3_bucket_public_access_block" "terraform_state" {
```

```
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}

# Create a DynamoDB table for Terraform state locking
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "terraform-httpd-state-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name    = "Terraform-State-Locks"
    Purpose = "Terraform State Locking"
  }
}
```

Final Complete Code (for Task 3,4,5) → output.tf File

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP address of the web server"
  value       = aws_instance.web.public_ip
}

output "website_url" {
  description = "URL of the Apache web server"
  value       = "http://${aws_instance.web.public_ip}"
}

output "terraform_state_bucket" {
  description = "S3 bucket used for Terraform state"
  value       = aws_s3_bucket.terraform_state.bucket
}
```

```
output "terraform_lock_table" {
  description = "DynamoDB table used for Terraform state locking"
  value       = aws_dynamodb_table.terraform_locks.name
}
```